Apache Arrow + Polars + MinIO + Parquet Hive

MinIO

Github Repo

MinIO es una solución de almacenamiento de objetos compatible con *S3* y de alto rendimiento, publicada bajo la licencia *GNU AGPL v3.0*. El almacenamiento de objetos es una **tecnología especialmente adaptada para su utilización en el** Data Lakehouse. Diseñada para ofrecer velocidad y escalabilidad, potencia la *IA/ML*, el análisis y las cargas de trabajo con un uso intensivo de datos con un rendimiento líder en el sector

Versión Arrow-Only

El código siguiente muestra cómo acceder **utilizando únicamente la integración de *Apache Arrow* con *Python*** a datos en un servidor ***S3 MinIO*** almacenados en formato *Parquet* y particionados al estilo *Hive*

```
import pyarrow as pa
import pyarrow.dataset as ds
import pyarrow.compute as pc

# Definir conexión a MinIO
minio = pa.fs.S3FileSystem(
     endpoint_override='minio:9000',
     access_key='YA9JokyWUb2hFUbKYEEN',
     secret_key='0k2ornkQpVTqUrBbOEsEXOnBEEWgJf4AFQOU4O7Y',
     scheme='http')

# Definir schema de los datos
schema = pa.schema([
    pa.field("tts", pa.timestamp('us')),
    pa.field("ServiceType", pa.string()),
    pa.field("ServiceDate", pa.string()),
    pa.field("ServiceTrain", pa.uint32()),
    pa.field("StopCode", pa.string()),
    pa.field("StopDepartureTime", pa.time64('us'), nullable=True),
    pa.field("StopDepartureDelay", pa.int64(), nullable=True),
    pa.field("StopArrivalTime", pa.time64('us'), nullable=True),
    pa.field("StopArrivalDelay", pa.int64(), nullable=True),
    pa.field("ns", pa.uint16())
```

```
])

# Crear dataset desde MinIO Hive Partitioned
ads = (
    ds.dataset("labmtf/EuroTrain/",
               filesystem=minio,
               format="parquet", partitioning=ds.partitioning(schema=schema,
flavor="hive"))
    .filter(
        pc.field('ServiceType').isin(
            ['EuroCity', 'Eurocity Direct', 'Eurostar', 'Nightjet', 'European
Sleeper', 'ICE International']
        )
    )
)

# Convertir en tabla Arrow y presentar resultado como Pandas Dataframe
ads.to_table(
    columns=["ServiceType", "ServiceDate", "StopCode", "StopArrivalTime",
"StopDepartureTime"],
    filter=pc.field("ServiceID") == "15070216"
).to_pandas()
```

	ServiceType	ServiceDate	StopCode	StopArrivalTime	StopDepartureTime
0	EuroCity	25-12-29	ATW	06:42:00	06:45:00
1	EuroCity	25-12-29	BD	07:18:00	07:26:00
2	EuroCity	25-12-29	BERCH	06:36:00	06:38:00
3	EuroCity	25-12-29	BRUSC	05:47:00	05:49:00
4	EuroCity	25-12-29	BRUSN	05:53:00	05:56:00
5	EuroCity	25-12-29	BRUSZ	None	05:44:00
6	EuroCity	25-12-29	FBNL	06:07:00	06:11:00
7	EuroCity	25-12-29	MECH	06:22:00	06:24:00
8	EuroCity	25-12-29	NDKP	06:59:00	07:01:00
9	EuroCity	25-12-29	RTD	07:50:00	None

Polars + Arrow: Scan a PyArrow dataset

A diferencia del ejemplo previo, en este se emplea un Polar LazyFrame para consultar los datos cargados desde *MinIO* mediante un *pyarrow.dataset*. En consecuencia, requiere añadir `import polars`

```
import pyarrow as pa
import pyarrow.dataset as ds
import pyarrow.compute as pc
import polars as pl

# Definir conexión a MinIO
```

```
minio = pa.fs.S3FileSystem(
     endpoint_override='minio:9000',
     access_key='YA9JokyWUb2hFUbKYEEN',
     secret_key='0k2ornkQpVTqUrBbOEsEXOnBEEWgJf4AFQOU4O7Y',
     scheme='http')

# Definir schema de los datos
schema = pa.schema([
    pa.field("tts", pa.timestamp('us')),
    pa.field("ServiceType", pa.string()),
    pa.field("ServiceDate", pa.string()),
    pa.field("ServiceTrain", pa.uint32()),
    pa.field("StopCode", pa.string()),
    pa.field("StopDepartureTime", pa.time64('us'), nullable=True),
    pa.field("StopDepartureDelay", pa.int64(), nullable=True),
    pa.field("StopArrivalTime", pa.time64('us'), nullable=True),
    pa.field("StopArrivalDelay", pa.int64(), nullable=True),
    pa.field("ns", pa.uint16())
])

# Crear dataset desde MinIO Hive Partitioned
ads = ds.dataset("labmtf/EuroTrain/", filesystem=minio, format="parquet",
partitioning=ds.partitioning(schema=schema, flavor="hive")).filter(
    pc.field('ServiceType').isin(['EuroCity', 'Eurocity Direct', 'Eurostar',
'Nightjet', 'European Sleeper', 'ICE International'])
)

lazy_df = (
    #
https://docs.pola.rs/api/python/stable/reference/api/polars.scan_pyarrow_datas
et.html
    pl.scan_pyarrow_dataset(ads).filter(pl.col("ServiceID") ==
"15070216").select(
        ["ServiceType", "ServiceDate", "StopCode", "StopArrivalTime",
"StopDepartureTime"]
    )
)

lazy_df.collect().to_pandas()
```

	ServiceType	ServiceDate	StopCode	StopArrivalTime	StopDepartureTime
0	EuroCity	25-12-29	ATW	06:42:00	06:45:00
1	EuroCity	25-12-29	BD	07:18:00	07:26:00
2	EuroCity	25-12-29	BERCH	06:36:00	06:38:00
3	EuroCity	25-12-29	BRUSC	05:47:00	05:49:00
4	EuroCity	25-12-29	BRUSN	05:53:00	05:56:00
5	EuroCity	25-12-29	BRUSZ	None	05:44:00
6	EuroCity	25-12-29	FBNL	06:07:00	06:11:00
7	EuroCity	25-12-29	MECH	06:22:00	06:24:00
8	EuroCity	25-12-29	NDKP	06:59:00	07:01:00

	ServiceType	ServiceDate	StopCode	StopArrivalTime	StopDepartureTime
9	EuroCity	25-12-29	RTD	07:50:00	None

Versión Polars-Only: Scan Parquet (Hive Partitioning)

Este último ejemplo es *Polars-Only*, no hace falta nada más. Como en los ejemplos precedentes, el dataset se almacena particionado en un servidor de objetos ***S3 MinIO***

```
import polars as pl

# Configuración del endpoint de MinIO y credenciales
storage_options={"aws_access_key_id": "YA9JokyWUb2hFUbKYEEN",
    "aws_secret_access_key": "0k2ornkQpVTqUrBbOEsEXOnBEEWgJf4AFQOU4O7Y",
    "endpoint_url": "http://minio:9000"
}

s3url = "s3://labmtf/EuroTrain/**/*.parquet"

# Dafinir LazyFrame
lazy_df = (
    pl.scan_parquet(s3url, storage_options=storage_options,
hive_partitioning=True)
    .filter(pl.col("ServiceID") == "15070216")
    .select(["ServiceDate", "ServiceType", "StopCode", "StopArrivalTime",
"StopDepartureTime"])
)

# Realizar operaciones y recolectar resultados
df = lazy_df.collect()
# Mostrar metadatos
print(df.glimpse(return_type="frame"))
# Mostrar resultado
df.to_pandas()
```

```
┌──────────────────┬───────┬──────────────────────────────────────┐
│ column           ┆ dtype ┆ values                               │
│ ---              ┆ ---   ┆ ---                                  │
│ str              ┆ str   ┆ list[str]                            │
╞══════════════════╪═══════╪══════════════════════════════════════╡
│ ServiceDate      ┆ str   ┆ ["'25-12-29'", "'25-12-29'", …… │
│ ServiceType      ┆ str   ┆ ["'EuroCity'", "'EuroCity'", …… │
│ StopCode         ┆ str   ┆ ["'ATW'", "'BD'", … "'RTD'"]     │
│ StopArrivalTime  ┆ time  ┆ ["06:42:00", "07:18:00", … "07… │
│ StopDepartureTime ┆ time  ┆ ["06:45:00", "07:26:00", … nul… │
└──────────────────┴───────┴──────────────────────────────────────┘
```

	ServiceDate	ServiceType	StopCode	StopArrivalTime	StopDepartureTime
0	25-12-29	EuroCity	ATW	06:42:00	06:45:00
1	25-12-29	EuroCity	BD	07:18:00	07:26:00

	ServiceDate	ServiceType	StopCode	StopArrivalTime	StopDepartureTime
2	25-12-29	EuroCity	BERCH	06:36:00	06:38:00
3	25-12-29	EuroCity	BRUSC	05:47:00	05:49:00
4	25-12-29	EuroCity	BRUSN	05:53:00	05:56:00
5	25-12-29	EuroCity	BRUSZ	None	05:44:00
6	25-12-29	EuroCity	FBNL	06:07:00	06:11:00
7	25-12-29	EuroCity	MECH	06:22:00	06:24:00
8	25-12-29	EuroCity	NDKP	06:59:00	07:01:00
9	25-12-29	EuroCity	RTD	07:50:00	None